ECE 484: Principles of Safe Autonomy Fall 2026

Written by: ECE 484 Team

MP 3: Kalman Filtering and Localization *Website*  *Due Date: 11/13/2026*

1 Introduction

You are tasked with developing the localization module for an autonomous drone racing pipeline. The drone will fly through a sequence of gates arranged in a known track. In other words, you will be provided with a known environment, where the positions and orientations of all gates are provided. You will develop the localization/state estimation module which will compute the position with respect with global coordinate system. This will be passive localization, i.e., the drone motion is controlled independently (not by your module).

The MP has two parts. First (Sec. 2), you will solve *theoretical problems about control and filtering*. In the second part (Sec. 3), you will implement a Kalman Filter for state estimation from IMU and camera. You will be provided Rosbags with monocular camera and odometry information. Using the image measurements of known gate positions, you will estimate the drone's pose in the global frame. In this pipeline, simulated VIO odometry provides the motion prior and PnP provides the position measurement for Kalman Filter fusion. The simulated VIO odometry emulates the output of a visual odometry system rather than providing ground-truth motion. Its pose and velocity estimates are imperfect and will accumulate drift over time. Therefore, it is important to use the Kalman Filter to reduce this drift and provide accurate state estimates.

The perception pipeline also depends on your Kalman Filter. To find the gates in each camera image, it predicts where each gate should appear using the current state estimate. A good estimate makes the gates easy to find, and finding the gates keeps the estimate accurate. No ground-truth information is used anywhere in the pipeline. In addition to the Kalman Filter, you will implement the parts of the image processing and PnP modules that connect the filter to the camera.

Learning objectives

- Kalman Filter
- Kalman Prediction
- Kalman Measurement Update
- Coordinate transformations and camera projection
- Using the state estimate to predict measurements
- Data association

System requirements

- Ubuntu 24.04
- ROS2 Jazzy

2 Written Problems

Problem 1 (20 points)**.** Consider the following two two-dimensional linear time invariant systems $\dot{x} = A_1 x$ and $\dot{x} = A_2 x$, where

$$A_1 = \begin{bmatrix} -1 & 4 \\ 2 & -3 \end{bmatrix}, \qquad A_2 = \begin{bmatrix} -1 & 4 \\ -2 & -3 \end{bmatrix}.$$

(a) Find the characteristic polynomial and eigenvalues of each matrix.
(b) Determine whether each system is asymptotically stable.
(c) A student claims that a continuous-time linear system is asymptotically stable if and only if every eigenvalue of its system matrix has magnitude less than 1. Is this claim TRUE or FALSE? Justify your answer.

Problem 2 (20 points)**.** Consider the following two-dimensional linear time invariant system

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u = Ax + Bu.$$

We would like to design a state-feedback controller to make the system asymptotically stable. Let the feedback law be of the form

$$u = -\begin{bmatrix} k & 4-k \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = -K(k)x, \qquad k \in \mathbb{R}.$$

Write down the closed-loop differential equations and determine all values of k for which the closed-loop system is asymptotically stable. Show your work.

Problem 3 (10 points)**.** Consider the linear system parameterized by $\alpha \in \mathbb{R}$:

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = x_3, \qquad \dot{x}_3 = -\alpha x_1 - 2x_2 - 3x_3.$$

(a) Derive the characteristic equation $\det(\lambda I - A(\alpha)) = 0$ for this system.
(b) Use the Hurwitz criteria to determine all values of α for which the system is asymptotically stable.
Hint: A monic cubic $\lambda^3 + a_1\lambda^2 + a_2\lambda + a_3$ has all roots in the open left half-plane if and only if

$$a_1 > 0, \qquad a_2 > 0, \qquad a_3 > 0, \qquad a_1 a_2 > a_3.$$

Problem 4 (20 points)**.** A robot navigates a 1D world consisting of five discrete positions. Landmarks are located at positions 2 and 4. The robot's sensor detects a landmark with probability 0.8 at landmark positions and 0.1 elsewhere.

A particle filter is initialized with three equally weighted particles at positions $[1, 3, 4]$. A control input moves each particle one position to the right with probability 0.7, or leaves it in place with probability 0.3.

For this problem, instead of randomly sampling a motion outcome, consider both possible outcomes for each particle, with their weights determined by the motion probabilities. After the motion update, the sensor detects a landmark.

(a) Determine all possible particle positions after the motion update and their corresponding weights.
(b) Compute the normalized weights of the resulting particles using the sensor measurement. Which position has the highest total weight?

Problem 5 (10 points)**.** Consider a robot with state $X \in \{F, O\}$. Let h denote the previously available information, and suppose

$$P(X = O \mid h) = p, \qquad P(X = F \mid h) = 1 - p, \qquad 0 < p < 1.$$

The robot receives a new measurement z. Assume that the measurement is conditionally independent of h given the state, i.e.,

$$P(z \mid X = s, h) = P(z \mid X = s), \qquad s \in \{F, O\}.$$

Let $L_O = P(z \mid X = O)$ and $L_F = P(z \mid X = F)$, with $L_O, L_F > 0$. Using conditional Bayes' rule, express the posterior odds

$$\frac{P(X = O \mid z, h)}{P(X = F \mid z, h)}$$

in terms of the prior odds $p/(1-p)$ and the likelihood ratio L_O/L_F. Show your derivation and explain how the measurement affects the belief when $L_O = L_F$.

Problem 6 (20 points)**.** In this problem, you will apply the Bayes filter over two time steps for a system with states $X_t \in \{F, O\}$. The transition model is

$$T = \begin{bmatrix} 3/4 & 1/2 \\ 1/4 & 1/2 \end{bmatrix}, \qquad T_{ij} = P(X_t = i \mid X_{t-1} = j),$$

where both rows and columns are in the order $[F, O]$. The initial belief, in the same order, is

$$\mathrm{bel}(x_0) = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}.$$

The measurement model is

$$P_M(\mathrm{hit} \mid F) = \tfrac{1}{4}, \qquad P_M(\mathrm{hit} \mid O) = \tfrac{3}{4},$$
$$P_M(\mathrm{miss} \mid F) = \tfrac{3}{4}, \qquad P_M(\mathrm{miss} \mid O) = \tfrac{1}{4}.$$

Assume the state process is Markov and measurements are conditionally independent given the state trajectory, with the time-t measurement depending only on X_t. At time 1, the robot receives $z_1 = \mathrm{hit}$. At time 2, the sensor packet is lost. Packet loss is independent of the state and sensor reading, so the loss event provides no information about the state.

(a) Compute $\mathrm{bel}(x_1)$ after receiving $z_1 = \mathrm{hit}$. Show the prediction, unnormalized measurement weights, and normalization.
(b) Compute $\mathrm{bel}(x_2)$ when the time-2 packet is lost. Show the prediction and explain how the loss event affects the update.

3 Implementing Kalman Filter to Localize in Racing Environment

3.1 Module architecture

This section describes components that are important for this MP. The components marked by * are not required for you to implement, but you will need to be familiar with them. The overall architecture of this MP is shown in Fig. 1. Detailed explanation for each of the components in the figure can be found in the following section.

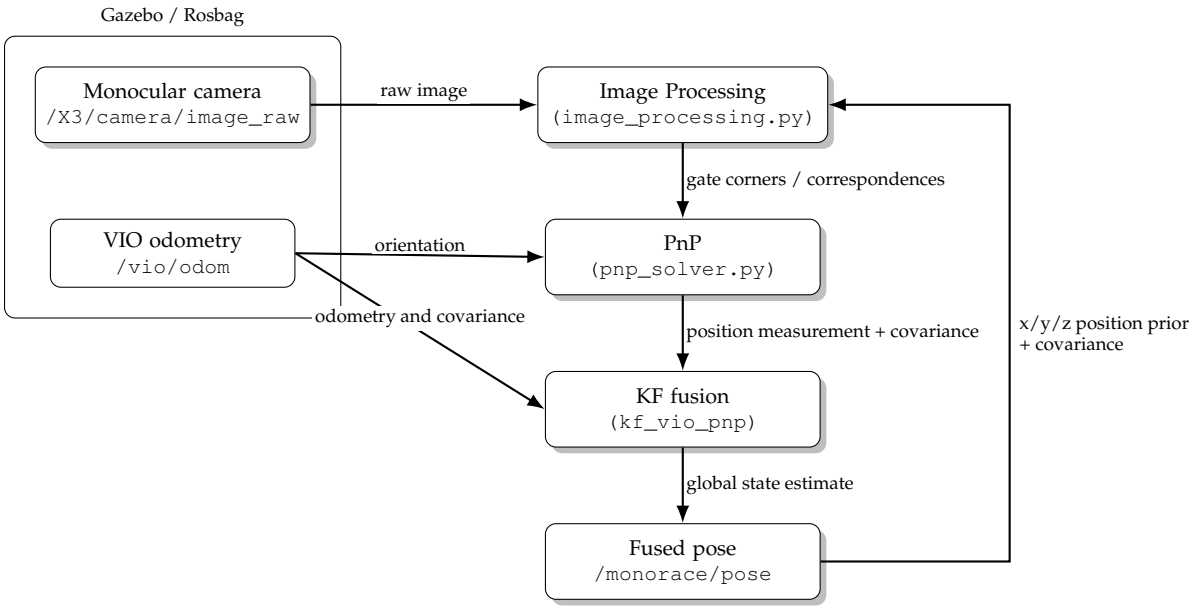


Figure 1: Architecture of this MP

3.1.1 Rosbag*

A Rosbag is a recorded file of ROS topic messages from a previous simulation run. You will play this Rosbag, which republishes the recorded topics in their original time order. This allows your pipeline to receive repeatable camera and odometry inputs across multiple runs. Your Kalman Filter implementation will be evaluated using the provided Rosbags, which contain both raw camera data and odometry measurements. Here, the `/vio/odom` emulates the output of a practical visual odometry system rather than provide ground-truth motion. As a result, its pose and velocity estimates are imperfect and may accumulate drift over time due to noise and estimation error.

3.1.2 Gazebo Simulator*

A Gazebo environment is provided that consists of the same gate layout used in the Rosbags. Your Kalman Filter will **not** be evaluated using the Gazebo simulator, but you will be able to control the drone in the Gazebo environment to conduct testing. The raw camera images as well as odometry measurements will be published by Gazebo.

3.1.3 Image Processing Module

This module is in `perception_student/perception/image_processing.py`. It takes monocular camera images, position priors, and known gate information in the world frame as input. It performs

image segmentation and basic filtering to prepare visual observations for PnP. Its output is a set of 2D pixel coordinates of gate corners, along with the corresponding 3D world coordinates of each gate corners.

To compute these, this module uses a prior pose: the drone's position from your Kalman Filter estimate (`/monorace/pose_cov`) and its orientation from VIO. It projects the known gate corners into the image from this prior pose, and then matches each detected gate to a gate in the map. The segmentation network and the corner detection are provided. You will implement the projection and the matching, as described in Sec. 3.2.

3.1.4 Perspective-n-Points (PnP) Solver Module

This module is in `perception_student/perception/pnp_solver.py`. This module takes filtered 2D pixel coordinates of gate corners to the 3D world coordinates correspondences as input, then solves for camera pose in the world frame, and feeds out pose estimates and quality metadata.

The PnP solver itself is provided. You will implement the camera extrinsics, and the conversion from the PnP solution to the drone pose, as described in Sec. 3.3. For more background, see the OpenCV `solvePnP` documentation.

3.1.5 Kalman Filter Fusion Module

This module is in `perception_student/kf_vio_pnp.py`. Implementation details for functions you will complete are in Sec. 3.4.

3.1.6 Coordinate Frames

The pipeline uses three coordinate frames:

- World frame W: the fixed frame of the gate map, with z pointing up.
- Base frame B: attached to the drone body, with x forward, y left, and z up.
- Camera frame C: the OpenCV camera frame, with z pointing out of the lens, x right, and y down.

T_{AB} denotes the 4×4 homogeneous transformation that maps coordinates in frame B to coordinates in frame A. Therefore, $\tilde{p}_A = T_{AB}\,\tilde{p}_B$.

3.2 Image Processing

The image processing module uses the estimated drone pose to project known gate positions from the map into the camera image. It compares these predicted pixel locations with detected gate locations in each image to identify which gate in the map each detection corresponds to. These matches provide the 2D image points and corresponding 3D map points needed by the PnP solver to estimate the drone's pose. Additionally, a position covariance of the current pose estimate is used to account for uncertainty when matching gates.

In `perception_student/perception/image_processing.py`, you will implement the following functions.

```
def project_points(pts_world, T_world_base, T_base_cam, K):
    # world points -> pixel coordinates
    ...

def project_position_uncertainty(point_cam, pos_cov_world, R_cam_world, K):
    # position uncertainty -> pixel uncertainty
```

```
    ...

def associate_detections(det_centroids, prior_centroids, prior_sigmas,
                         base_sigma_px=15.0, gate_threshold=3.0):
    # match detected gates to predicted gates
    ...
```

3.2.1 `project_points()`

This function predicts where known 3D points should appear in the camera image using the estimated drone pose. It transforms world-frame points into the camera frame using the drone pose T_{WB} and camera extrinsics T_{BC}, then projects them into pixel coordinates using the intrinsic matrix K. This function returns both the pixel coordinates and the corresponding camera-frame points. Do **not** use `cv2.projectPoints`.

3.2.2 `project_position_uncertainty()`

Since the prior position of the drone is uncertain, the predicted pixel coordinates in the image are uncertain as well. This function converts the position covariance of the prior into a pixel uncertainty for a predicted point given in the camera frame.

To do this, linearize the pinhole projection

$$u = f_x \frac{X}{Z} + o_x, \qquad v = f_y \frac{Y}{Z} + o_y$$

around the predicted camera-frame point $p_c = [X\ Y\ Z]^\top$. Compute the 2×3 Jacobian

$$J = \begin{bmatrix} \frac{\partial u}{\partial X} & \frac{\partial u}{\partial Y} & \frac{\partial u}{\partial Z} \\ \frac{\partial v}{\partial X} & \frac{\partial v}{\partial Y} & \frac{\partial v}{\partial Z} \end{bmatrix},$$

evaluating each derivative at p_c. With J, the 2×3 Jacobian of the projection with respect to the camera-frame point, and Σ_W, the position covariance in the world frame, the covariance in the image can be computed by

$$\Sigma_{\text{px}} = J R_{CW}\, \Sigma_W\, R_{CW}^\top J^\top. \tag{1}$$

Return the largest standard deviation along any direction in the image, which is the square root of the largest eigenvalue of Σ_{px}, or infinity if p_c is not in front of the camera.

3.2.3 `associate_detections()`

The segmentation network and corner detector find gates in the image, but do not identify which gate in the map each detection corresponds to. This function matches detected gate centers in the image to gate centers projected onto the image from the 3D map using the estimated drone pose. Both sets of centers are expressed in pixel coordinates.

For each detected gate d and projected map gate p, compute the matching cost

$$c_{dp} = \frac{\|c_d - c_p\|_2}{s_p}, \qquad s_p = \sqrt{\sigma_p^2 + \sigma_{\text{base}}^2}. \tag{2}$$

Here, c_d is the detected gate center from `det_centroids`, and c_p is the projected gate center from `prior_centroids`. The prediction uncertainty σ_p is the corresponding entry in `prior_sigmas`,

computed by `project_position_uncertainty()`. The detection uncertainty σ_{base} is given by `base_sigma_px`, which defaults to 15 pixels. If σ_p is infinite, set $c_{dp} = \infty$.

When the predicted pixel location is reliable, a detected gate should be close to it to be considered a match. When the prediction is less certain, a larger separation may still be reasonable. Dividing the pixel distance by s_p accounts for this uncertainty when comparing possible matches.

Construct the cost matrix containing all c_{dp} values, then repeatedly take the lowest-cost pair whose detection and map gate are both still unmatched. Each detection and each projected map gate may appear in at most one match.

After computing the assignment, retain only matches satisfying

$$c_{dp} \leq \texttt{gate_threshold},$$

This rejects matches whose pixel separation is too large relative to their combined uncertainty.

Return the accepted matches as (detection index, gate index) pairs, where the indices refer to `det_centroids` and `prior_centroids`, respectively.

3.3 PnP Solver

You will implement the following functions in `perception_student/perception/pnp_solver.py`.

```
def build_T_base_cam(t_base_cam, pitch):
    # camera extrinsics T_base_cam
    ...

def body_pose_from_pnp(rvec, tvec, T_base_cam):
    # PnP solution -> drone pose T_world_base
    ...
```

3.3.1 `build_T_base_cam()`

This function computes the camera extrinsics T_{BC}, the transformation from the camera frame to the base frame. The camera is mounted at position `t_base_cam` in the base frame and rotated by `pitch` (in radians) about the base y-axis; a negative pitch tilts the camera up. The matrix `R_LINK_OPTICAL` in `pnp_solver.py` maps coordinates in the camera frame C into the camera mount frame.

3.3.2 `body_pose_from_pnp()`

The PnP solver returns a rotation vector `rvec` and a translation vector `tvec` in the same format as OpenCV's `solvePnP`; see its documentation for which transformation they represent. The Kalman Filter needs the position of the drone body rather than the camera. This function converts the PnP solution into the drone pose T_{WB} in the world frame.

3.4 Kalman Filter

You will implement a linear Kalman Filter (KF) that fuses VIO priors with position measurements from PnP. Although the VIO odometry is not perfect and may accumulate drift over time, its uncertainty can be modeled in the Kalman filter framework as Gaussian noise with an associated covariance. This allows the VIO output to be incorporated as a probabilistic prior during state fusion. In this Kalman Filter, the state vector is

$$x_t = [p_x\ p_y\ p_z\ v_x\ v_y\ v_z]^\top \in \mathbb{R}^6$$

Where the first three components represent position and the last three represent velocity. The prediction input is VIO velocity, and the correction input is position-only PnP measurement

$$z_{\text{pnp}} = [p_x \; p_y \; p_z]^\top \in \mathbb{R}^3$$

The Kalman filter assumes that the system evolves according to a linear Gaussian state-space model:

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t,$$
$$z_t = C_t x_t + \delta_t,$$

where A_t is the state transition matrix, B_t is the control-input matrix, and C_t is the observation matrix. The process noise ϵ_t and measurement noise δ_t are assumed to be zero-mean Gaussian:

$$\epsilon_t \sim \mathcal{N}(0, Q_t), \qquad \delta_t \sim \mathcal{N}(0, R_t),$$

where Q_t and R_t denote the process and measurement noise covariance matrices, respectively[1]. Under these assumptions, the posterior estimate of the state remains Gaussian.

Therefore, the Kalman filter represents the state estimate using a mean μ_t and covariance Σ_t. These mean and covariances will be continuously updated as VIO odometry measurements are published. When PnP measurements are published, the state will be corrected accordingly. This is represented in the flowchart below.

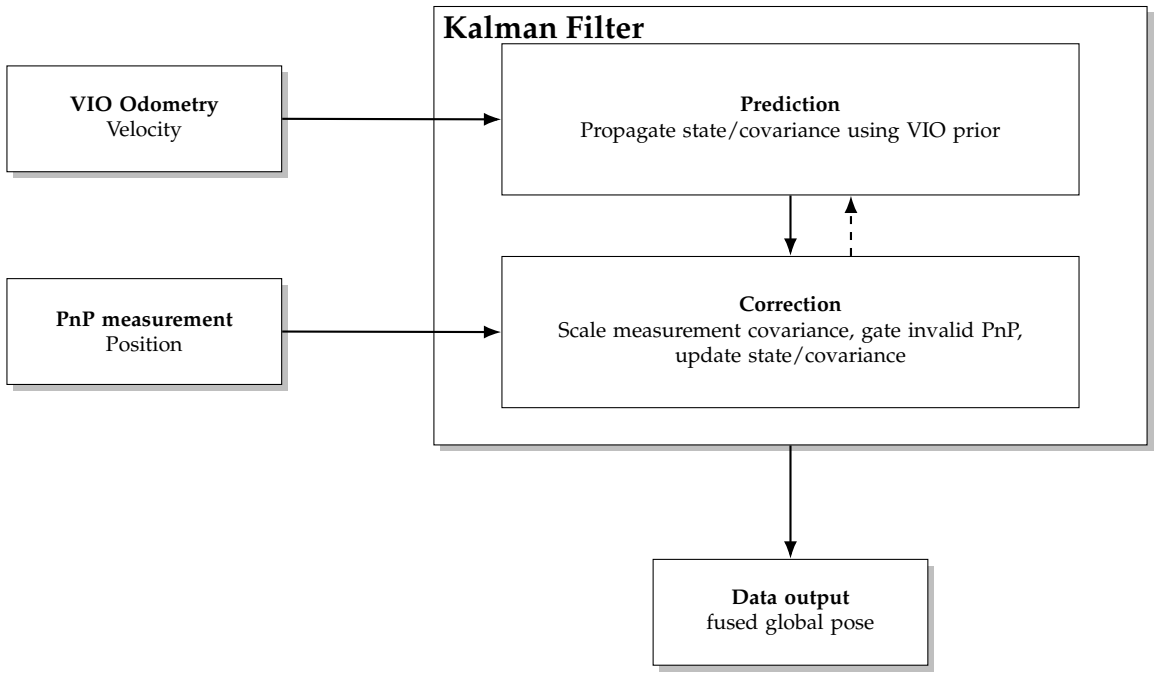


Figure 2: Flowchart of Kalman Filter Algorithm.

You will implement the core filter logic in `perception_student/kf_vio_pnp.py` (class `VioPnpKF`).

[1]Notation may vary across sources; some textbooks and lecture slides use the opposite convention, assigning Q_t to measurement noise and R_t to process noise.

```
class VioPnpKF:
    def predict_vio(time_sec, vio_velocity, vio_vel_cov):
        # prediction: propagate state/covariance with VIO
        ...

    def update_pnp(z_pos, R_pos):
        # correction: apply PnP position update
        ...
```

3.4.1 `predict_vio()`

Implement this function in `perception_student/kf_vio_pnp.py`. This function takes VIO velocity as input and performs the standard Kalman Filter prediction step from lecture: it propagates the state mean through the motion model, and propagates the state covariance through the state transition matrix while adding the process noise covariance:

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t, \qquad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^\top + Q_t. \tag{3}$$

Here, μ_t is the state mean estimate, Σ_t is the state covariance, A_t is the state transition matrix, B_t is the input matrix, u_t is the VIO velocity prior, and Q_t is the process noise covariance.

In this predictor, the position is propagated by integrating the latest VIO velocity over the time step Δt, and the velocity state is replaced by the latest VIO velocity. Derive A_t and B_t from this model.

The process noise covariance Q_t represents uncertainty introduced during one prediction step. Construct it from the VIO velocity variance and how this velocity affects the predicted position and velocity over Δt.

3.4.2 `update_pnp()`

Implement this function in `perception_student/kf_vio_pnp.py`. This function takes a PnP position measurement and measurement covariance as input and performs the standard Kalman Filter correction step from lecture: it computes the Kalman gain from the predicted covariance and the measurement noise covariance, corrects the predicted mean using the innovation, and reduces the covariance accordingly:

$$\mu_t = \bar{\mu}_t + K_t \left(z_t - C_t \bar{\mu}_t \right) \tag{4}$$

$$\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t \tag{5}$$

$$K_t = \bar{\Sigma}_t C_t^\top \left(C_t \bar{\Sigma}_t C_t^\top + R_t \right)^{-1} \tag{6}$$

Here, C_t is the measurement matrix, R_t is the measurement noise covariance, K_t is the Kalman gain, and z_t is the PnP position measurement.

The innovation is the difference between the measured position and the predicted position. Before applying the correction, it is recommended to reject outliers using the Mahalanobis distance of the innovation, which normalizes by the innovation covariance S_t:

$$d_t^2 = y_t^\top S_t^{-1} y_t, \qquad y_t = z_t - C_t \bar{\mu}_t, \qquad S_t = C_t \bar{\Sigma}_t C_t^\top + R_t. \tag{7}$$

This test tolerates large residuals when uncertainty is large and rejects them when uncertainty is small. If d_t^2 exceeds a threshold (suggested initial value of 11.345, corresponding to a 3-DoF χ^2 gate near 99%), the measurement is rejected as an outlier, and the predicted state and covariance are kept; otherwise, the KF correction is applied.

3.4.3 Robustness Tuning

You may also tune the covariance matrices in the Kalman filter to improve performance and accuracy. Currently, the PnP measurement noise is computed according to this paper. You can modify the `sigma_pos_base` value to modify the PnP covariances in `perception_student/perception/pnp_measurement.py`.

Additionally, you can modify the initial state covariance in the Kalman Filter file, under `initialize()` as well. This covariance determines how much the filter trusts its initial state estimate. A larger initial covariance indicates less confidence in the initial state estimate, while a smaller initial covariance indicates greater confidence.

3.5 Running the Experiment

Navigate to the `code/release` directory. Create a virtual environment and install Python dependencies with

```
python3 -m venv --system-site-packages ~/mp3_venv
source ~/mp3_venv/bin/activate
pip install -r src/perception_student/requirements.txt
```

Activate the virtual environment in each new terminal as well. Then, build and source your workspace. Be sure to source in each new terminal.

```
colcon build --symlink-install
source install/setup.bash
```

3.5.1 Running with Rosbag

Download the Rosbags provided here. Unzip the zip files into the `code/release` directory. In order to play the a bag, run the following command while in the `code/release` directory

```
ros2 bag play <BAG NAME> --topics /X3/camera/image_raw \
    /X3/camera/camera_info /X3/odom /vio/odom
```

Replace `<BAG NAME>` with your bag name.

3.5.2 Running with Gazebo

Your pipeline will **not** be evaluated on the Gazebo simulator. This Gazebo environment contains the same gate configuration as the one use in the Rosbags. The simulator is provided for your convenience to test individual portions of the racing track. First, launch the Gazebo simulator.

```
ros2 launch mav_bringup mav_init.launch.py
```

If you have launched the simulator correctly, you should see the following Gazebo window.

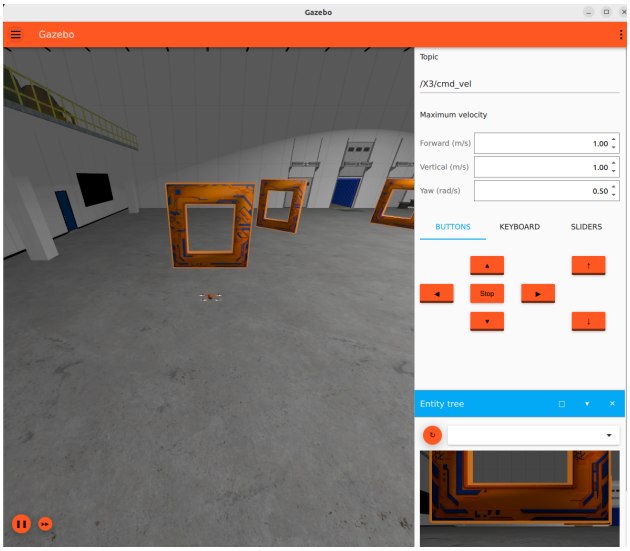


Figure 3: RViz Screen After Launch

Click on the 'Keyboard' tab in order to manually control the drone with W A S D keys.

3.5.3 Running the Rest of the Pipeline

Once you have chosen either a Rosbag or Gazebo as your source of sensor input, then you can launch the Kalman Filter stack in a new terminal. If you are playing a Rosbag, run the following command with the flag `use_bag:=true`

```
ros2 launch perception_student perception_student.launch.py
# or, when playing a Rosbag:
ros2 launch perception_student perception_student.launch.py use_bag:=true
```

If this launch is successful, you should see the following RViz Window.

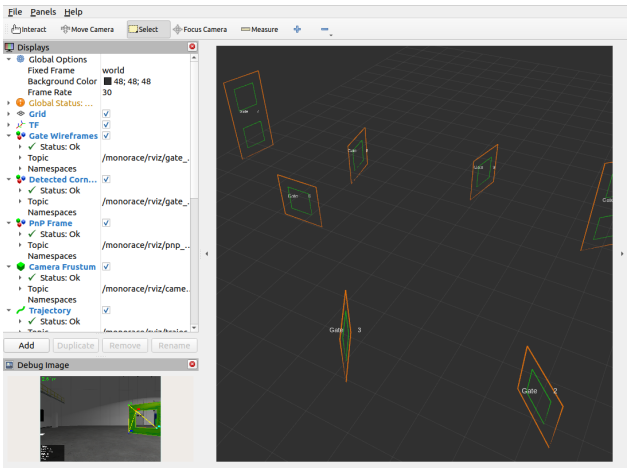


Figure 4: RViz Screen After Launch

In order to launch trajectory visualization and X/Y/Z error graphs, run the following commands in a new terminal.

```
ros2 run perception_student trajectory_plotter
```

You should be able to see the following windows.

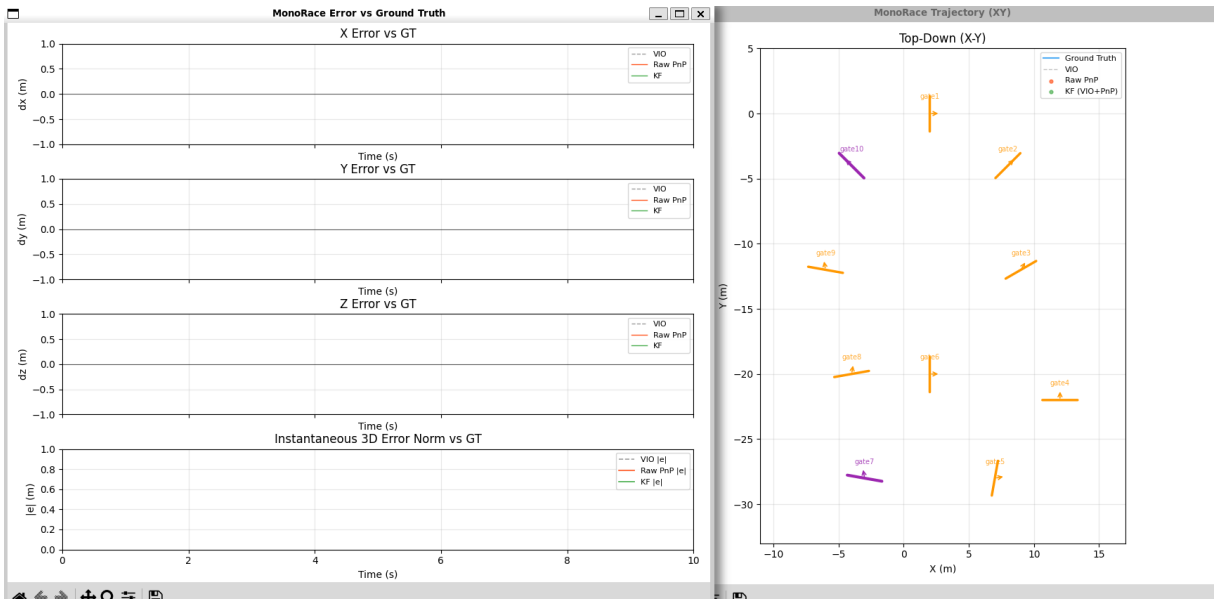


Figure 5: Error and Trajectory Plots

3.6 Metrics

For this MP, you should strive to minimize the Root Mean Square Error (RMSE) between the Kalman Filter estimated pose and the ground truth pose of the drone. Your Instantaneous 3D error is expected to stay

under 0.2m for 90% of the full run, and under 0.3m for 95% of the full run. Your final RMSE error should be below 0.175m.

3.7 Report

For the MP3 report, you will need to answer the following problems

Problem 7 (20 points)**.** Record videos of the KF pipeline running with all Rosbags. **Make sure links to the videos are accessible and not private!**

Problem 8 (15 points)**.** Include the X/Y/Z position error plots as well as the top-down trajectory plots for all Rosbags

Problem 9 (10 points)**.** Based on your X/Y/Z error plots and top-down trajectory plots, discuss how the fused estimate compares against the raw inputs. What types of errors are reduced by the KF?

Problem 10 (15 points)**.** If the fused estimate performs poorly on one Rosbag but well on another, what are some plausible reasons? Discuss at least two factors related to sensing, noise, or filter assumptions.

Problem 11 (10 points)**.** The correction step typically produces an estimate with lower uncertainty than either the prediction or the measurement alone. Why does combining two uncertain sources often produce a more confident estimate?

Problem 12 (10 points)**.** In the MP, a Kalman Filter estimate is used to predict where gates appear in the image. How does the KF covariance affect gate association, and what would go wrong if the covariance were much smaller than the true error?

Problem 13 (20 points)**. Demo:** Demonstrate full pipeline

4 Report and Submission

Problems 1-6 must be done individually (Homework 3). Write solutions to each problem in a file named `hw3_<netid>.pdf` and upload the document in Gradescope. If you use other sources, cite them and say how you used them. We strongly discourage using AI for this part.

Problems 7–13 can be done in groups. Each group should write a report that contains the solutions, plots, and discussions. This report should be submitted to Gradescope. Please include links to the video in the report.